PROJECT BLUEPRINT

ATLAS

AI Financial Assistant for Telegram

A conversational financial analyst that lives in your pocket —
built for the Atlas AI Financial Assistant Hackathon.

PREPARED FOR	Atlas AI Financial Assistant Hackathon
DOCUMENT TYPE	End-to-end architecture, product & build roadmap
TARGET PLATFORM	Telegram (text, voice, images)
CORE VERTICAL	Finance — research, briefings, documents, alerts
PRIZE TRACK	Founding Engineer — Hackathon Winner

"I understand how you work — let me do it for you."

Generated build reference — not affiliated with any specific vendor. Model and API choices are recommendations.

Table of Contents

1 Executive Summary

2 Product Vision & Design Principles

3 Target User & Core Promise

4 Experience Map — What the User Actually Feels

5 Conversation Design — Onboarding & Natural Dialogue

6 System Architecture

7 Tech Stack

8 Database Schema

9 AI / Agent Architecture

10 Feature Modules

11 Integrations Plan

12 Proactive Intelligence Engine

13 Voice & Image Handling

14 Security, Privacy & Reliability

15 Build Roadmap (5-Day Sprint)

16 Evaluation Alignment Scorecard

17 Demo Video Script

18 Risks & Mitigations

19 Stretch Features (Post-MVP Wow Factor)

1 Executive Summary

Atlas is a Telegram-native AI financial assistant designed to feel like texting an experienced analyst who already knows your portfolio, your calendar, and your inbox. It replaces the daily ritual of tab-switching between market data sites, news feeds, spreadsheets, and email with a single conversational thread that remembers context, learns preferences over time, and reaches out proactively only when something genuinely matters.

The product is built around three commitments that map directly to the hackathon's evaluation weights: it must be useful every single day (30%), it must demonstrate sharp product judgment about what *not* to build (25%), it must converse like a person rather than a command-line tool (20%), and it must go deep on finance rather than wide across verticals (15%).

This document is the complete build reference: architecture, data model, AI design, integration plan, and a day-by-day roadmap for shipping a working bot and demo video.

What makes it stand out

- **Silence as a feature.** The daily brief engine is built to say nothing on quiet days — explicitly scored against 'quality over frequency.'
- **Memory that compounds.** Every conversation writes structured facts (tickers followed, risk tolerance, preferred brief time) into a persistent profile the agent re-reads every turn.
- **One brain, many senses.** Text, voice notes, and document photos all resolve to the same internal request format, so the assistant behaves identically no matter how you talk to it.
- **Clarify before guessing.** Ambiguous requests ('tell me about Apple') trigger a short follow-up instead of a wall of text nobody asked for.

2 Product Vision & Design Principles

The brief is explicit that this is not a chatbot, not a ChatGPT wrapper, not a news reader. Atlas is designed around a single test applied to every feature before it gets built:

> *“Does this save the user time today, or is it a feature for the demo reel?”*

Five principles enforced throughout the build:

- **Concise by default.** No response exceeds what fits on a phone screen without scrolling; depth is available on request, never forced.
- **Signal over noise.** Every proactive message must pass a 'so what?' filter — it explains impact, not just fact.
- **Conversational, not command-driven.** No slash commands, no inline keyboards, no menus. Natural language is the entire interface.
- **Progressive personalization.** Onboarding asks the minimum; the agent keeps learning from every subsequent message.

- **Finance-first.** All engineering hours before Day 4 go to the finance vertical. Additional verticals are a stretch goal only.

3 Target User & Core Promise

Primary persona: a busy analyst, investor, or founder who checks markets throughout the day across five or six different tools. Secondary persona: a finance student or early-career professional who wants research help without a Bloomberg terminal.

Persona	Pain Today	Atlas Promise
Analyst / PM	Fragmented research across Bloomberg, email, news, filings	One thread, one memory, sourced answers
Founder	No time to track macro or competitor news	Silent unless it matters, then explains why
Student / Early career	Doesn't know what to monitor or how to read filings	Document Q&A; + plain-English explanations

4 Experience Map — What the User Actually Feels

Judges will interact with the live bot, so the emotional arc of the first five minutes matters as much as the architecture behind it.

Time	Moment	What happens
0:00	Open chat	Bot sends a warm, human opener — no wall of text, no form.
0:15	Two or three light questions	Role, tickers/sectors of interest, preferred brief time — asked conversationally, skippable anytime.
0:45	Optional connect	Bot mentions Gmail/Calendar/Drive naturally, offers a one-tap connect, easy to defer.
1:00	First value moment	Bot immediately returns something real: today's move on a ticker the user just mentioned.
Day 1, 8:00am	Morning brief	Short, personalized, only sent if something is genuinely worth surfacing.
Ongoing	Natural asks	User asks questions like they would text a colleague; bot clarifies when ambiguous.
Ongoing	Quiet proactivity	Alerts fire only on real thresholds (earnings, filings, % moves) the user set or implied.

5 Conversation Design — Onboarding & Natural Dialogue

5.1 Onboarding script (conversational, not a form)

Implemented as a lightweight state machine layered on top of the LLM — the state machine tracks which facts are still missing, but the LLM phrases every question in natural language and reacts to whatever the user actually says, including answers that jump ahead or skip questions entirely.

- **Opener:** greet, state what Atlas does in one sentence, ask what to call them.
- **Role:** "What best describes what you do — investor, analyst, founder, something else?"
- **Focus:** "Any companies, sectors, or tickers you want me watching from day one?"
- **Insight type:** "When I flag something, should I lean toward market news, earnings, filings, or macro?"
- **Cadence:** "Want a morning brief? What time works?"
- **Optional connect:** "I can read your calendar to help you prep for meetings, or your inbox to track deal threads — want to connect either now, or later?"

Every question accepts 'skip', silence, or a topic change as a valid answer — the state machine simply marks the field unknown and moves on. Users can start asking real questions mid-onboarding and the bot answers immediately, folding onboarding into the background.

5.2 Handling ambiguity

Every incoming request passes through an intent classifier before any tool runs. If the request matches more than one likely intent with comparable confidence, the agent asks a single, specific clarifying question instead of guessing.

> ***User:*** *"Tell me about Apple"*
> ***Atlas:*** *"Sure — want the latest news, the financial picture, how the stock is trading, or a general company overview?"*

Clarification is capped at one question per turn and never asked for requests that are already unambiguous — the brief calls this out as a UX quality signal, not a way to stall.

5.3 Memory in every reply

Before generating a response, the agent loads: (a) the rolling conversation summary, (b) the structured user profile, (c) any watchlist or alert state, and (d) the last few raw turns. This context is injected into the system prompt on every call, so the assistant never re-asks something it was already told and can reference prior discussions naturally ("like we talked about last week when NVDA reported").

6 System Architecture

The system is a modular backend fronted by a single Telegram bot. All channels — text, voice, images — normalize into one internal message format before hitting the agent orchestrator.

High-level flow

Telegram Update → Webhook Ingest → Channel Normalizer (STT / OCR / text passthrough) → Agent Orchestrator (intent + memory + tools) → Tool Layer (financial data, docs, integrations, web search) → Response Composer (concise formatting) → Telegram Send.

In parallel, a Scheduler service runs independently of user messages to power the Proactive Intelligence Engine (briefs, alerts, watchers).

Core services

Component	Responsibility
Telegram Gateway	Receives webhooks, sends replies, handles voice/photo downloads.
Channel Normalizer	Whisper for voice → text; GPT-4o Vision / Claude Vision + OCR for images → text/structured data.
Agent Orchestrator	LLM-driven router: classifies intent, decides tools, manages clarification, assembles context.
Memory Service	Reads/writes user profile, watchlists, running conversation summary, embeddings for long-term recall.
Tool Layer	Typed functions the LLM can call: get_quote, get_filing, search_news, read_email, read_calendar, analyze_sheet, summarize_document, set_alert.
Integration Adapters	OAuth-backed clients for Gmail, Calendar, Drive, Sheets; REST clients for financial data providers.
Scheduler / Worker	Cron-style jobs: morning brief generation, watchlist polling, alert threshold checks, follow-up reminders.
Response Composer	Enforces brevity rules, formats for Telegram (Markdown), attaches sources.
Data Layer	PostgreSQL (structured) + pgvector or a vector store (semantic memory / document chunks).

Why this shape

- Normalizing every input channel to one internal format means the agent logic is written once, not three times — voice and photo requests get the same reasoning quality as typed ones.
- Separating the Scheduler from the request/response loop lets proactive intelligence run independently, which is essential for a bot that must speak first without being asked.

- A typed Tool Layer keeps the LLM auditable — every action it takes is a named function call with logged inputs/outputs, not a free-form script.

7 Tech Stack

Layer	Choice	Why
Bot Framework	Telegraf.js (Node) or python-telegram-bot (FastAPI)	Mature, webhook-first, active community
Backend	Node.js + NestJS *or* Python + FastAPI	Modular, DI-friendly, easy background jobs
Primary LLM	Claude (Sonnet-class) via Anthropic API, tool use enabled	Strong tool-calling + long context for memory
Speech-to-Text	Whisper (OpenAI) or Deepgram	Reliable for short voice notes
Vision / OCR	Claude Vision or GPT-4o Vision + Tesseract fallback	Reads financial document photos/screenshots
Database	PostgreSQL	Relational integrity for profiles, watchlists, alerts
Vector Store	pgvector (in Postgres) or Pinecone	Semantic recall over past chats & documents
Queue / Scheduler	BullMQ (Redis) or APScheduler	Briefs, polling, reminders as background jobs
Financial Data	Finnhub + SEC EDGAR + Alpha Vantage (fallback)	Prices, fundamentals, filings, earnings calendar
Web Search / News	Tavily or Bing Search API + source de-duplication	Live synthesis with citations
Productivity APIs	Google OAuth: Gmail, Calendar, Drive, Sheets	Meeting prep, email context, spreadsheet analysis
Hosting	Railway / Render / Fly.io for MVP; Docker Compose locally	Fast iteration, cheap always-on webhook
Observability	Sentry + structured JSON logs	Debuggable tool-call chains during judging

Model choice is explicitly not an evaluation criterion — this stack favors Claude for its tool-use reliability and long context, but any strong function-calling model (GPT-4o, Gemini 1.5/2.0) is a drop-in swap behind the Tool Layer abstraction.

8 Database Schema

Minimal, normalized schema covering everything the brief requires: profiles, conversation history, preferences, integrations, documents, and memory.

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  telegram_id BIGINT UNIQUE NOT NULL,
  display_name TEXT,
  role TEXT,                    -- investor / analyst / founder / student / other
  onboarding_state TEXT,        -- in_progress / complete / skipped
  brief_time TIME,              -- preferred daily brief time
  timezone TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE preferences (
  user_id UUID REFERENCES users(id),
  key TEXT,                     -- 'insight_type', 'verbosity', 'risk_tolerance'...
  value JSONB,
  updated_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, key)
);

CREATE TABLE watchlist_items (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  ticker TEXT,
  sector TEXT,
  reason TEXT,                  -- why the user is tracking it (inferred or stated)
  added_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE alerts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  ticker TEXT,
  condition TEXT,               -- 'price_move_pct', 'sec_filing', 'earnings_date'
  threshold JSONB,
  active BOOLEAN DEFAULT TRUE,
  last_triggered_at TIMESTAMPTZ
);

CREATE TABLE conversations (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  role TEXT,                    -- user / assistant / system
  channel TEXT,                 -- text / voice / image
  content TEXT,
  tool_calls JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE memory_facts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  fact TEXT,                    -- "Prefers concise morning briefs before 8am"
  embedding VECTOR(1536),
  source_conversation_id UUID REFERENCES conversations(id),
  created_at TIMESTAMPTZ DEFAULT now()
);
```

```
CREATE TABLE integrations (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  provider TEXT,                -- gmail / calendar / drive / sheets
  access_token_enc TEXT,
  refresh_token_enc TEXT,
  scopes TEXT[],
  connected_at TIMESTAMPTZ
);

CREATE TABLE documents (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  source TEXT,                  -- telegram_upload / drive
  title TEXT,
  doc_type TEXT,                -- 10-K / earnings_deck / model / other
  raw_text TEXT,
  summary TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE briefs_sent (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  brief_type TEXT,              -- morning / evening / alert
  content TEXT,
  sent_at TIMESTAMPTZ DEFAULT now()
);
```

Design notes: **memory_facts** stores distilled, embeddable statements rather than raw chat logs, so semantic recall stays cheap and relevant. **briefs_sent** exists specifically to prevent duplicate or redundant proactive messages — a quiet day should produce zero rows.

9 AI / Agent Architecture

Atlas uses a single orchestrating agent with a typed tool set, rather than a rigid multi-bot pipeline — this keeps conversation feeling like one consistent personality while still allowing deep, specialized reasoning per request.

9.1 Turn lifecycle

- **Normalize:** voice → text (Whisper), image → text/structured data (Vision + OCR).
- **Load context:** user profile, watchlist, last N turns, relevant memory_facts via embedding search.
- **Classify intent:** research / document Q&A; / scheduling / alert management / small talk / ambiguous.
- **Clarify if needed:** single targeted question, then wait for the reply before proceeding.
- **Plan & call tools:** the LLM selects one or more tools (parallel where possible: e.g. quote + news).
- **Compose:** Response Composer enforces length caps, adds sources, strips fluff.
- **Write-back:** new facts extracted from the exchange are embedded and stored in memory_facts.

9.2 Tool Layer (function-calling contract)

Tool	Purpose
get_quote(ticker)	Live price, day change, volume — Finnhub/Alpha Vantage
get_fundamentals(ticker)	Revenue, margins, valuation multiples
get_filings(ticker, type)	SEC EDGAR filings, insider transactions
get_earnings_calendar(range)	Upcoming earnings dates for watchlist
search_news(query, window)	Live web/news synthesis with source list
summarize_document(doc_id, question)	RAG over uploaded/Drive documents
compare_entities(a, b, criteria)	Structured side-by-side financial comparison
read_email(query)	Gmail search + thread summarization (scoped, user-approved)
read_calendar(range)	Upcoming meetings, attendee/company context
analyze_sheet(sheet_id, question)	Google Sheets anomaly/KPI analysis
set_alert(ticker, condition)	Registers a watcher for the Scheduler
schedule_reminder(when, content)	One-off reminder, e.g. before an earnings call

9.3 Prompting discipline

- System prompt encodes tone (analyst, not chatbot), brevity rules, citation requirements, and the explicit instruction to ask before assuming on ambiguous requests.

- Every financial claim must trace to a tool result — the model is instructed to say 'I don't have confirmed data on that' rather than infer numbers.
- A lightweight verbosity controller trims responses to the shortest form that fully answers the question, with an offer to expand ('want the full breakdown?').

10 Feature Modules

10.1 Company & Market Research

- Company profile, financials, recent news, leadership changes, funding/M&A; — synthesized, not linked.
- Side-by-side comparisons (e.g. Microsoft vs Google) across revenue growth, margins, valuation.
- Every answer explains *why it matters*, not just what happened.

10.2 Financial Document Intelligence

- Upload a 10-K, earnings deck, or model as a file or photo; ask questions conversationally.
- RAG pipeline: chunk → embed → retrieve → answer with page/section references.
- Multi-document compare ("how did this quarter's deck change vs last quarter's").

10.3 Live Financial Information

- Prices, filings, earnings, macro events pulled live when local knowledge is stale or missing.
- Confidence is stated explicitly — the assistant flags when it can't verify a number rather than guessing.

10.4 Meeting & Email Intelligence (Google integrations)

- Calendar-aware meeting prep: pulls attendee company context automatically before a call.
- Gmail thread summarization for a given company/deal, with action items extracted.
- Follow-up reminders generated from commitments made in email threads.

10.5 Spreadsheet Intelligence (Sheets/Drive)

- Natural-language analysis of a linked sheet: KPI trends, anomaly detection, forecast vs actual gaps.
- Executive summary generation from a raw model without manual formatting.

11 Integrations Plan

Integrations are chosen for depth of value, not count — each one is justified against a concrete user moment rather than added for the feature list.

Integration	Job to be done	Concrete use case	Auth
Gmail	Meeting prep	Summarize all threads with a company before a call	OAuth read-only, scoped search
Google Calendar	Daily awareness	Surface today's meetings with relevant company context	OAuth read-only
Google Drive	Document access	Pull financial models/decks the user already has, no re-upload	OAuth read-only

Integration	Job to be done	Concrete use case	Auth
Google Sheets	Model analysis	Conversational KPI/anomaly review on live spreadsheets	OAuth read/write optional
SEC EDGAR	Ground truth filings	10-K/10-Q/8-K retrieval, insider transactions	Public API, no auth
Finnhub	Market data	Real-time-ish quotes, earnings calendar, company news	API key
Alpha Vantage	Fallback data	Redundancy if Finnhub rate-limits during judging	API key
Web/News Search	Freshness	Breaking news synthesis with citations	API key

All Google integrations are introduced conversationally during or after onboarding, are entirely optional, and can be connected later with a single natural-language request ('connect my calendar').

12 Proactive Intelligence Engine

This is the component most likely to differentiate Atlas from a 'ChatGPT wrapper' — it is a background system that decides, independently, whether today is worth interrupting the user for.

Pipeline

- **Collect:** scheduler pulls watchlist prices, filings, earnings calendar, and top macro headlines for each active user at their configured brief time.
- **Filter:** a scoring pass ranks items by materiality (price move %, filing type, headline reach) against the user's stated risk tolerance and follow list.
- **Threshold check:** if nothing clears the bar, *no message is sent* — logged as a deliberate no-op, not a bug.
- **Synthesize:** surviving items are compressed into 3–5 lines max, each stating impact, not just fact.
- **Deliver & learn:** engagement (reply, ignore, 'more detail please') feeds back into future materiality scoring for that user.

Example output

> *"Morning — NVDA is up 4.2% pre-market after a beat-and-raise from a major customer; that's the clearest read-through to your AI/semis watchlist this week. Tesla's 8-K filed overnight is routine (exec compensation), nothing actionable. Markets otherwise quiet."*

Alert types supported out of the box: price-move thresholds, new SEC filings on watched tickers, earnings-date reminders (with a configurable lead time, e.g. 1 hour before a call), and user-defined natural-language watchers ('notify me if Tesla makes a major announcement').

13 Voice & Image Handling

13.1 Voice messages

- Telegram voice note → downloaded → transcribed via Whisper → treated as a normal text turn.
- Replies stay text by default (Telegram-native, scannable); voice-reply is a stretch feature, not core.

13.2 Images

- Screenshots of charts/spreadsheets/filings routed to a vision model for structured extraction.
- Scanned or photographed documents get an OCR pass (Tesseract) as a fallback for low-quality images.
- Extracted content flows into the same summarize_document tool used for uploaded PDFs, so the experience is identical regardless of how the document arrived.

14 Security, Privacy & Reliability

- OAuth tokens encrypted at rest (KMS-backed or libsodium sealed boxes); never logged.
- Google scopes requested are read-only wherever the feature allows it, minimizing blast radius.
- Every financial figure is source-tagged internally so provenance can be shown or audited on request.
- Rate-limit and cache financial API calls (short TTL on quotes, longer on fundamentals) to protect against provider throttling mid-demo.
- Graceful degradation: if a data provider is down, the assistant says so plainly instead of fabricating a number — matches the brief's explicit accuracy requirement.
- All tool calls and LLM traces logged (redacted) for debugging judge sessions in real time.

15 Build Roadmap (5-Day Sprint)

Sequenced so that a genuinely working, demoable bot exists by the end of Day 2 — every day after that adds depth without risking a broken core.

Day	Theme	Deliverables
Day 1	Foundation	Telegram webhook + FastAPI/NestJS skeleton; Postgres schema migrated; Claude tool-calling loop wired to a stub tool; basic text round-trip working end to end.
Day 2	Core conversation + first data tool	Onboarding state machine; memory read/write; get_quote + search_news tools live via Finnhub + web search; ambiguity clarification logic.
Day 3	Research depth + documents	get_fundamentals, get_filings (EDGAR), compare_entities; document upload → RAG pipeline; image OCR/vision path for photographed documents.
Day 4	Proactivity + integrations	Scheduler service; morning brief pipeline with materiality filter; Google OAuth (Calendar + Gmail) with meeting-prep and thread-summary tools; alerts (set_alert).
Day 5	Polish, reliability, demo	Voice note transcription; response-length tuning; error handling/fallbacks for every external API; load a realistic multi-day persona for the demo; record the demo video; write submission notes.

If time runs short, cut in this order without damaging the core score: Sheets analysis → multi-document compare → voice replies → secondary verticals. Never cut: clarification logic, the silent-brief behavior, or source citation — these are the most visible product-judgment signals to a judge.

16 Evaluation Alignment Scorecard

Every major feature is mapped back to the stated weighting so effort is spent where it is actually scored.

Criterion	How Atlas addresses it
Usefulness, proactivity, value (30%)	Morning/evening briefs, watchlist alerts, meeting prep, document Q&A; — all designed around real daily workflows, not demo tricks.
Product thinking & judgment (25%)	Explicit silence rule on quiet days; no slash commands or menus; single clarifying question instead of guessing; finance-first scope discipline.
AI experience & conversational quality (20%)	Persistent memory across turns; natural-language onboarding; tone tuned to 'analyst', not 'assistant bot'; graceful ambiguity handling.
Depth of finance vertical (15%)	Live quotes, fundamentals, SEC filings, earnings calendar, document intelligence, comparisons — all finance-native, not generic Q&A.;
Engineering quality (10%)	Modular services, typed tool contracts, Postgres schema, background scheduler decoupled from chat loop, encrypted OAuth tokens, observability.

17 Demo Video Script

Target length: 3–4 minutes. Structure to show breadth without narrating architecture.

Timestamp	Beat	What to show
0:00–0:30	Cold open on a real problem	"Before Atlas, my morning looked like six tabs. Here's what it looks like now."
0:30–1:00	Onboarding	Fast cut through a natural onboarding conversation, show skip behavior.
1:00–1:45	Natural research ask	"Compare Microsoft and Google as investments" → concise, sourced answer.
1:45–2:15	Document intelligence	Upload an earnings deck photo; ask a specific question; get a cited answer.
2:15–2:45	Proactivity	Show a real morning brief; explicitly narrate that a quiet day produced no message the day before.
2:45–3:15	Integration moment	Ask for meeting prep pulling calendar + email context automatically.
3:15–3:45	Voice + close	Ask a question by voice note; close on the product's one-line promise.

18 Risks & Mitigations

Risk	Mitigation
Financial API rate limits during judging	Cache aggressively; Alpha Vantage as automatic fallback; pre-warm cache for demo tickers.
LLM hallucinating figures	Hard rule: no number without a tool result; explicit 'uncertain' fallback language.
OAuth setup friction for judges	Ship a pre-connected demo account; live judges can still test the skip path fully.
Over-scoping into secondary verticals	Hard gate: no non-finance vertical work until Day 5, and only if core is fully stable.
Proactive messages feeling spammy	Materiality threshold tuned conservatively; log every no-op day as a visible metric for the demo.
Voice/OCR accuracy on noisy input	Always show the transcribed/extracted text back to the user for an implicit confirmation.

19 Stretch Features (Post-MVP Wow Factor)

Only pursued once the finance core is fully solid — these are what turn 'complete' into 'extraordinary.'

- **Portfolio intelligence:** connect a Sheet as a live portfolio; brief adapts to actual holdings.
- **Sentiment overlay:** lightweight aggregate sentiment score alongside news synthesis.
- **Weekly retrospective:** Sunday-evening summary of the week's biggest moves across the watchlist.
- **Cross-vertical mode:** optional Startup Ecosystem or Business lens for users who select it, reusing the same memory and tool architecture — added only after finance is demo-ready.
- **Explainable alerts:** tapping into why a threshold fired, with the underlying data shown inline.

End of blueprint. This document is intended as a living build reference — update the schema, tool list, and roadmap as implementation decisions are finalized.